Understanding

Fourier neural operators

through fluid prediction

Chalmers AI Society
CAIS
RND branch

Måns Westman
Sofia Nguyen
October 2026

This report is the result of our society's project to implement a Fourier neural operator and understand how it works. We hope that this can be a valuable resource for anyone that wants to learn about neural operators or scientific machine learning in general as we have opted for a report centered around intuition. We thank our partner Aixia for providing the AiQu computing resources used to train the model.

CHALMERS AI SOCIETY CAIS

PROJECT PARTNER

aixio

Contents

1. Introduction **2**

2. Predicting fluid dynamics **3**
2.1. A fluid state as a field . . . 3
2.2. Saved frames and solver substeps . . . 3

3. From matrices to operators and kernels **4**
3.1. Why call it an operator? . . . 5
3.2. Turning a weight matrix into an integral kernel . . . 6
3.3. Convolution and displacement . . . 7
3.4. Why Fourier space speeds it up . . . 7
3.5. Putting a full Fourier block together . . . 9

4. Architecture and implementation from scratch **10**
4.1. Inputs and layers . . . 10
4.2. Building a forecast . . . 13
4.3. Zero-shot resolution transfer . . . 13

5. Data, training and evaluation **13**
5.1. Dataset and split . . . 13
5.2. Normalisation . . . 14
5.3. Loss and metrics . . . 15
5.4. Training setup . . . 16

6. Results **17**
6.1. Accuracy . . . 17
6.2. Speed versus accuracy . . . 18
6.3. Transfer to 256×256 without retraining . . . 19
6.4. Batching . . . 21

7. Discussion and conclusion **21**

Acknowledgements **21**

Appendix A. How the numerical solver works **22**
A.1. Why Fourier space simplifies viscosity . . . 22
A.2. The time-step update . . . 22
A.3. A geometric picture of explicit-step instability . . . 23

Appendix B. Batching and profiling **23**

Appendix C. Detailed solver comparisons **24**
C.1. The 64×64 comparison . . . 25
C.2. The 256×256 comparisons . . . 25
C.3. Implementation details . . . 25

References and further reading **26**

1. Introduction

There are many engineering problems that require predictions of how a quantity varies across space and changes over time. A temperature distribution or perhaps the deformation of a solid are two examples. Partial differential equations (PDEs) describe these relationships using derivatives with respect to position and time. To solve these PDEs and understand how the fluid or heat changes, numerical solvers turn the equations into calculations on a finite grid, often moving the solution forward through many small time steps.

Running these simulations can take a lot of computation, especially when we need to repeat them many times. For instance, when comparing designs or forecasting several possible outcomes we may need to solve a similar problem repeatedly but with different inputs. However, a trained surrogate model offers us an alternative approach. We can use examples produced by a numerical solver to train a model and then use that model to make further predictions. Of course, generating the examples and training the model still require computation, but the potential benefit comes from reusing the trained model across enough predictions to justify that initial cost.

Fourier neural operators (FNOs) are one architecture for this purpose. They learn relationships between entire fields, using a Fourier representation to exchange information across the spatial domain, from one part of the system to another. Applications of FNOs and their extensions include weather forecasting, fusion plasma stabilization, ocean modelling and predicting stress and strain in solids. Section 3.8 of Duruisseaux, Kossaifi and Anandkumar takes a closer look at some of these examples. Our project studies a much smaller problem that makes the underlying ideas easier to understand and examine. We try to predict the evolution of a two-dimensional fluid based on its history.

The aim of this report is to build intuition for FNOs and connect their mathematics to an implementation. We assume no prior knowledge of fluid dynamics or functional analysis. We first explain fields, operators and kernels, then show why Fourier representations are useful and how the resulting model is trained. Finally, we examine what our implementation achieves in accuracy, computational cost and transfer to a finer grid.

The specific model is an FNO-2d implemented in PyTorch without a high-level neural-operator library. We used the architecture introduced by Li et al. as our starting point. Given ten observed fluid states, the model predicts the next state and feeds that prediction back into its input. Repeating this process produces a forecast of twenty states. We compare it with a numerical solver at several time-step sizes and test whether weights trained on a 64×64 grid can also be used at 256×256.

Our focus will be on developing intuition and providing a first look at the capabilities of Neural Operators. For a more extensive treatment, we recommend *Fourier Neural Operators Explained: A Practical Perspective*, by Duruisseaux, Kossaifi and Anandkumar. It covers mathematical background and Fourier transforms, implementation, training and architectural extensions in much more detail. We believe that it is a tremendous resource for developing a deeper understanding of the field and also to explore other architectural variations. It helped us a lot while building our implementation, especially for understanding the Fourier analytical aspects of the model.

2. Predicting fluid dynamics

2.1. A fluid state as a field

Imagine placing a small wheel with paddles sticking out in every direction at discrete points in a moving fluid. Now, it is easy to imagine these wheels rotating at different velocities or different directions across the fluid. This is a good intuitive picture of **vorticity**. In two dimensions, vorticity is a positive or negative (signed) scalar: its sign distinguishes the two directions of rotation, and its magnitude measures the intensity or speed of the local rotational motion. A vorticity plot therefore shows one number at each spatial location. It does not actually display the direction or speed at which the fluid is travelling.

We write this field as $\omega(t, \mathbf{x})$, where t is time and $\mathbf{x} = (x_1, x_2)$ is a point in the domain. At a fixed time, $\omega(t, \cdot)$ means the whole spatial function, i.e., the actual fluid state at one point in time. A 64×64 array stores samples of that field at 4,096 locations. The most important mental model is to think about it as a continuous function with peaks and valleys that is sampled at discrete points along a grid so that each point indicates "how much the fluid rotates" and in which direction.

Our domain is the periodic unit square $D = [0, 1]^2$. However, opposite edges connect, so this means that motion leaving the right edge continues from the left, and the top connects to the bottom (from a topological perspective, this makes it a torus! Try to imagine how connecting the edges would look). There are no solid walls or obstacles. Viscosity is fixed at $\nu = 10^{-4}$, and a fixed forcing pattern continually stirs the fluid (so that it doesn't just die out). Different trajectories begin from different (randomized) initial conditions and in that way the full trajectories differ vastly and give the surrogate model a wide training set.

The fluid dynamics theory might be difficult to understand in the first reading for those of you unfamiliar with PDEs, but having heard the concepts once will help with the following chapters and the understanding of why the architecture looks the way it does. To model the evolution of a fluid we can use the vorticity we introduced earlier:

$$\underbrace{\frac{\partial \omega}{\partial t}}_{\text{change at a fixed point}} + \underbrace{\mathbf{u} \cdot \nabla \omega}_{\text{transport by the flow}} = \underbrace{\nu \Delta \omega}_{\text{viscous smoothing}} + \underbrace{f}_{\text{external forcing}}$$

Here $\mathbf{u}$ is the velocity of the fluid, while $\nabla \omega$ describes how vorticity varies as we move through space. The term $\mathbf{u} \cdot \nabla \omega$ describes **advection**, which means that the moving fluid carries vorticity from one place to another. Because each paddlewheel is fixed at one grid point rather than floating with the water, it can suddenly start spinning faster not because any water is being stirred up, but simply because "calm" water has flowed away and an existing patch of high vorticity has drifted to its spot. The advection term accounts for this transport across the grid.

The Laplacian, $\Delta \omega = \partial^2 \omega / \partial x_1^2 + \partial^2 \omega / \partial x_2^2$, appears in the viscosity term, which smooths spatial variations in vorticity. Finally, f represents the external stirring. However, our network does not calculate these terms separately. Instead, it tries to capture the underlying dynamics by watching many examples of how the fluid evolves over time.

2.2. Saved frames and solver substeps

A **trajectory** is a sequence of fluid states that we have sampled with equally spaced intervals from one initial state. We use ten consecutive saved frames as input and the following twenty as targets. The saved frames are separated by one time unit, which we denote by $\Delta T = 1$.

(Sidenote: if you have heard about the recent breakthrough by OpenAI on the Navier–Stokes Millennium Prize Problem, the idea of an initial state appears there too. They also consider how a fluid evolves from an initial state, but ask whether a smooth three-dimensional flow can develop a singularity. However, in this project we only want to predict how our two-dimensional fluid evolves).

The numerical solver uses much smaller internal steps, denoted by Δt. It switches between physical space, where values are associated with positions, and Fourier space, where the same field is represented by waves. This is called a pseudo-spectral method because it combines calculations in the two representations. Spatial derivatives are convenient to calculate using waves, while the multiplication involved in advection is easier to perform using values at grid points. The solver switches between them to use each where it helps. Appendix A explains the update in more detail, (it gets quite involved but it is very interesting nonetheless!).

For advection, the solver uses the current flow to estimate how vorticity changes over the next small time step. The state it calculates then becomes the starting point for the following step, so it repeatedly uses its own approximate results. Errors can accumulate as the simulation advances, so if the step is too large, those errors can grow until the simulation becomes unstable.

At 64×64, the largest stable step we tested was $\Delta t = 1.6 \times 10^{-3}$ and when the timestep was 2.0×10^{-3} the solver diverged. Even at the larger stable step, the solver needs 625 substeps per saved interval and 12,500 for the twenty-frame forecast.

The FNO, on the other hand, predicts one saved frame in a single forward pass. It therefore uses twenty passes for the same forecast (!). This difference explains the opportunity for faster prediction, but those of you familiar with surrogate solvers in general may also see that the next important question is regarding the accuracy and cost of achieving those fast predictions. (Stay tuned for the final results).

3. From matrices to operators and kernels

Before looking at the individual layers, it helps to consider what we want the network to learn. We want it to predict an entire fluid field, with information from one part of the flow able to influence predictions elsewhere. We also want the learned rules to remain useful if we sample that field on a finer grid. Having this ability means that the cost of training a model on a new problem can be reduced as we can decrease the resolution during training and then evaluate on a higher resolution during inference.

Connecting every grid point to every other point would quickly become computationally expensive. This chapter follows how we can describe those connections using kernels, reuse the same interaction pattern through convolution, and calculate the result efficiently using Fourier transforms. We then look at how nonlinear activations let the network learn more complicated relationships than these weighted combinations alone.

One picture will be used throughout the chapter. We can imagine a field as a sheet of values over the square. Inside the network, we work with several such sheets of learned features, formed by combining the inputs. Each sheet is called a channel, and looking through the stack at one location gives a vector of feature values. Remember that these features generally do not represent actual physical quantities. Instead it can be helpful to think of them as intermediate values that the network learns to use and that lets it express more complex interactions.

Chapter 4 walks through how these ideas are actually implemented in the network, and we will revisit the key concepts from this chapter there. The connection between the math and the implementation usually becomes clearer as you go, so it might be helpful to return to anything

that felt tricky on a first pass.

3.1. Why call it an operator?

A fully connected network (MLP) acting on a flattened grid (stacking the discrete values of the field into a vector) maps one vector of fixed length to another:

$$f_\theta : \mathbb{R}^n \longrightarrow \mathbb{R}^m,$$

where θ are the learned weights. If we want to predict how a fluid evolves on a 64×64 grid, the obvious approach is to flatten the grid into a vector of 4096 numbers and train a network to predict the next 4096 numbers.

However, the fluid isn't actually a 4096-vector. It is a continuous field, and the grid is just where we happened to place our sensors (remember the paddlewheels) to measure the vorticity. When we flatten those measurements into a list, points that sit next to each other in the fluid can end up far apart in the list. The network does not automatically know that they are close to each other on the actual grid.

Because a fully connected network is shaped by the vector length $n = 4096$, it is fixed to that specific grid. If we try to pass a 128×128 grid of the exact same fluid, with the same behaviour, the input now has 16,384 numbers and the matrix multiplications don't even match in size.

To build a model that isn't tied to one grid resolution, we need a mapping that takes a function as input and outputs a function, that is the state of entire fluid field. In mathematical terms, a mapping between functions is called an operator. Differentiation ($\mathcal{D}[g] = g'$) is a classic linear operator: you feed in the function $g(x) = x^2$, and it returns a new function, $2x$.

A function space is a collection of functions with some specific properties (think of it as a vector space where the vectors are entire functions, which we can add together and multiply by scalars). For example, all functions of the form $g(x) = a + bx$ form a space described by two coefficients. However, more general continuous functions can not always be described exactly using a fixed finite number of coefficients, so the spaces we consider here are infinite-dimensional.

We can still do familiar things with these functions, such as adding them together or measuring how different two of them are. Later, our error metric will compare predicted and reference fields in this way. The computer works with grid samples (basically like finite representations of these infinite-dimensional objects), but the true mathematical object we are trying to describe is the whole function. [1]

This might sound quite complicated, but in essence, solving the Navier–Stokes equations takes a starting fluid state and gives us its state at a later time. We call this mapping a solution operator. Our model takes in a history of ten states and learns to approximate the next-state prediction. You can think of those ten vorticity frames as a single history function,

$$a_t(\mathbf{x}) = \big(\omega(t - 9\Delta T, \mathbf{x}), \ldots, \omega(t, \mathbf{x})\big),$$

predicting the fluid one time step in the future is therefore an operator that maps that history to the next vorticity field:

$$\mathcal{G} : a_t \longmapsto \omega(t + \Delta T, \cdot).$$

[1] For those curious about the terminology used in the papers in this field, this leads into normed spaces and Banach spaces. A Banach space has a notion of size (norm) and is complete. Completeness meaning that sequences whose members become arbitrarily close to one another (are Cauchy) have a limit within the space. But we will not need this theory here!

At any single point $\mathbf{x}$, $a_t(\mathbf{x})$ is a vector of ten numbers (the spin history at that spot), and across the whole domain, it is a stack of ten sheets with the past ten states of the fluid. Following the framework of Kovachki et al. (2021), a neural operator learns an approximation $\mathcal{G}_\theta$ and the parameters θ are independent of how finely we make the grid as it predicts the state of the field. It is like a rule that applies to all points, not just those at the coordinates where we provided samples.

3.2. Turning a weight matrix into an integral kernel

So predicting fields instead of vectors using operator theory does sound like a good idea for our purposes, but you might ask yourself "How do you actually build a neural network layer that acts on a function instead of a fixed vector?". The core idea is to take a standard linear layer and replace its discrete weight matrix with a continuous integral. Since we want to learn how values at different locations contribute to an output, if we can find a rule that applies everywhere we can evaluate wherever our grid points happen to be!

In a normal linear layer over M grid points, the output at pixel i is a weighted sum over all input pixels j:

$$z_i = \sum_{j=1}^{M} A_{ij} v_j.$$

Choose one output point i. To calculate its value, we multiply each input value v_j by its weight A_{ij} and add the contributions. Row i of the matrix A contains all the weights for this one output point. Moving to the next row gives the weights for the next output point.

In the matrix, we used the indices i and j to get the weight connecting two grid points. Now suppose we want to find that weight using their physical positions on a grid instead. We introduce a function $\kappa(\mathbf{x}, \mathbf{y})$ that takes an output position $\mathbf{x}$ and an input position $\mathbf{y}$, and gives us the weight connecting them. This function is called a kernel and it can be evaluated on any two points of the field. Adding the weighted contributions from across the continuous domain gives

$$(Kv)(\mathbf{x}) = \int_D \kappa(\mathbf{x}, \mathbf{y}) v(\mathbf{y})\, d\mathbf{y}.$$

For each output position $\mathbf{x}$, the integral adds up the input values across the domain, each multiplied by the weight given by the kernel.

When we actually compute this with a uniform grid of M pixels (where each pixel covers a small area ΔA), the integral turns into (or rather can be approximated by) a Riemann sum:

$$(Kv)(\mathbf{x}_i) \approx \sum_{j=1}^{M} \kappa(\mathbf{x}_i, \mathbf{x}_j) v(\mathbf{x}_j)\, \Delta A.$$

So the matrix entries we introduced earlier were really $A_{ij} \approx \kappa(\mathbf{x}_i, \mathbf{x}_j)\Delta A$. It is also important to point out the role of the area ΔA, if we were to switch to a finer grid with $4\times$ as many pixels, we would be adding up $4\times$ as many numbers, which would blow up the sum unless each pixel's contribution is scaled down by its smaller area ΔA. So effectively ΔA means that with a finer grid we just compute a sharper approximation of the exact same integral.

The important intuition to take away from this section is that if we start with a single sheet of numbers, then $v(\mathbf{y})$ is the value at position $\mathbf{y}$, and the kernel $\kappa(\mathbf{x}, \mathbf{y})$ gives the weight connecting that position to $\mathbf{x}$. For example, if the weights are strongest when $\mathbf{y}$ is near $\mathbf{x}$, nearby values contribute most to the output there. If the weights extend across the domain, distant values contribute too. Positive and negative weights allow contributions to reinforce or cancel one another, very similar to the weights in an ordinary neural network layer.

For now, let us stay with one sheet. To calculate the output at one position, we need to get the weighted contributions from every input position. We then repeat this for every position where we want an output. On a 64×64 grid, there are 4096 positions, so this already gives

$$\underbrace{4096}_{\text{output positions}} \times \underbrace{4096}_{\text{input positions}} \approx 16.8 \text{ million contributions.}$$

Inside the network, we have several stacked sheets of features, and values from different sheets can contribute to each other too (we will develop this more in Chapter 4). With several stacked sheets there are even more combinations to calculate.

We therefore want a way to connect all parts of the field without calculating all these contributions individually. This leads us to convolution and Fourier transforms.

3.3. Convolution and displacement

This is one of the clever solutions that FNOs present. We can make the problem a lot easier if we assume the interaction between two points only depends on their relative displacement $\mathbf{x} - \mathbf{y}$, meaning how far apart they are and in what direction, instead of their actual location:

$$\kappa(\mathbf{x}, \mathbf{y}) = k(\mathbf{x} - \mathbf{y}).$$

Suppose we move the output point one grid cell to the right. If the weights depend only on relative displacement, the whole weighting pattern moves with it without changing. Applying the same shifted pattern at every output location is the operation we call convolution,

$$(Kv)(\mathbf{x}) = \int_D k(\mathbf{x} - \mathbf{y}) v(\mathbf{y})\, d\mathbf{y},$$

So instead of learning a rule for every different location $\mathbf{x}$, we slide the same interaction rule k across the whole domain (Intuition: think about the sliding window of Convolutional Neural Networks). However, it is important to remember that we are not just on the real line, and because $\mathbf{x} - \mathbf{y}$ is a vector, direction still has to be taken into account (a point to the left can be weighted differently from a point to the right).

A standard CNN layer uses this same weight-sharing idea, usually over a small window such as 3×3 pixels. Imagine making that window larger until it includes the whole domain. Now even distant points can contribute and on our periodic square, the pattern "wraps around" wherever it crosses an edge.

This is very useful for our fluid problem because the behaviour at one point depends on what is happening at other locations in the fluid. So the network needs to connect information from across the fluid to predict what happens next.

3.4. Why Fourier space speeds it up

Reusing the same weighting rule means that we have fewer weights to learn, but we still have to apply that pattern at every output position. For one sheet with M grid points, we still gather M contributions at each of the M output positions (remember, M is total grid points not just the height or width). Calculating a global convolution directly therefore takes $O(M^2)$ operations.

Fourier transforms give us a way to calculate the same result with much less work. Using the Fast Fourier Transform (FFT), the cost drops to $O(M \log M)$! In this section we will give a brief introduction to how this works.

Instead of describing a sheet by listing its value at every pixel $\mathbf{x}$, we describe it as a sum of sine and cosine waves across the square. On a periodic unit square, each wave is labelled by a pair of integer frequencies $\mathbf{k} = (k_1, k_2)$ that count how many full oscillations the wave makes along the two coordinate directions. If you are unfamiliar with spatial frequencies, low frequencies describe broad variations or "large waves" across the square, while high frequencies describe closely spaced oscillations. In our vorticity field, these help describe the large patterns but also the finer ripples. Using Euler's formula,

$$e^{i\phi} = \cos\phi + i\sin\phi, \qquad i^2 = -1,$$

we express the cosine and sine parts of each wave into a single complex number $\widehat{v}(\mathbf{k})$ (aka the Fourier coefficient) which stores how strong/large that wave is (amplitude) and how far it is shifted (phase). Multiplying this coefficient by a complex weight lets us change both of those things without changing the wave's frequency.

3.4.1. Convolution preserves frequency

So back to our question, why does describing the field as a sum of waves help with convolution? If the input is a single sine wave, a convolution can change its amplitude and phase, but its frequency stays the same. This provides a shortcut. We can now calculate the convolution frequency by frequency. As we will see later, we can remove some of these frequencies so that the actual calculations are far fewer. In this subsection we will provide a brief intuition of why this is the case using one spatial dimension to keep the notation simple (but the same idea applies to our waves that run across our square).

Suppose our input is $v(y) = \sin(2\pi n y)$. Here, n is a positive integer giving the number of full waves across our periodic unit interval. The convolution gives

$$z(x) = \int k(x-y)\sin(2\pi n y)\,dy.$$

Now write the displacement as $d = x - y$, so $y = x - d$. We have not changed the input, only how we describe the positions we are adding over:

$$z(x) = \int k(d)\sin\big(2\pi n(x-d)\big)\,dd.$$

Using the familiar identity for the sine of a difference, we get

$$z(x) = \underbrace{\left[\int k(d)\cos(2\pi n d)\,dd\right]}_{A}\sin(2\pi n x) - \underbrace{\left[\int k(d)\sin(2\pi n d)\,dd\right]}_{B}\cos(2\pi n x).$$

This looks a little involved, but for a fixed kernel and frequency, A and B are just two numbers. So the output is a sine and a cosine with the same frequency. Together, these form a single wave with a different amplitude and phase:

$$z(x) = A\sin(2\pi n x) - B\cos(2\pi n x) = r\sin(2\pi n x - \phi).$$

The weighting can therefore change the height of the wave and move its peaks, but it cannot change how far apart they are. This works because the kernel depends only on displacement, making A and B the same at every output position.

The same reasoning applies to each wave across our two-dimensional square, where the frequency is labelled by $\mathbf{k} = (k_1, k_2)$. A field contains many waves, and because convolution is linear, we can work out what happens to each wave separately. Instead of calculating contributions from every

grid point to every other point, we can just multiply each wave's coefficient by the corresponding coefficient of the kernel. This is what is known as the convolution theorem. For a single sheet,

$$\widehat{Kv}(\mathbf{k}) = \widehat{k}(\mathbf{k})\,\widehat{v}(\mathbf{k}),$$

where hats denote Fourier coefficients.

We can now see why this representation provides such a shortcut. Instead of computing a global spatial sum where all M output points must talk to all M input points, costing $O(M^2)$, we move to the frequency domain. Because frequencies do not mix during convolution, the convolution just becomes a scalar multiplication per frequency coefficient (for each channel C). The FFT handles all conversions across the grid in $O(M \log M)$ time, and this lets us compute the global convolution at a fraction of the cost.

3.4.2. Learning the frequency weights

In the actual network, as we have mentioned earlier, the input fields are first turned into several sheets of learned features. We will follow that step in Chapter 4. For now, imagine that we have C such sheets and use the FFT to find the frequency coefficients of each one.

Pick one frequency, for example a wave that makes two full oscillations across the square in the horizontal direction. Each of our C sheets has a coefficient for that frequency, so we have C numbers to work with. To produce the coefficient in one output sheet, we take a learned weighted sum of those numbers, much like an ordinary neural-network layer. We do this for each of the C output sheets. That gives us C weights for each of C outputs, which we collect in a $C \times C$ matrix called $R(\mathbf{k})$. Each frequency has its own matrix, so the network can combine the sheets differently for broad waves and finer ones.

$$\widehat{Kv}_o(\mathbf{k}) = \sum_{c=1}^{C} R_{oc}(\mathbf{k})\widehat{v}_c(\mathbf{k}),$$

where c and o index the input and output channels. Because we learn the matrices $R(\mathbf{k})$ directly in Fourier space, we never actually construct or store the spatial kernel $k(\mathbf{x} - \mathbf{y})$. Basically, the kernel helped us to understand what we want to do, and then we utilize the Fourier transform to get a faster way to do it (but it is still the same convolution). Then we use the inverse FFT to get back feature sheets again from the modified waves, which gives us actual values at discrete grid points again. We can write the whole operation as

$$Kv = \mathcal{F}^{-1}[R\,\mathcal{F}[v]]\,.$$

Because every Fourier wave stretches across the entire square, multiplying by our weight matrix $R(\mathbf{k})$ and running the inverse FFT ($\mathcal{F}^{-1}$) updates every pixel on the grid at once. Notice, though, that $R(\mathbf{k})$ only mixes channels at the same frequency $\mathbf{k}$, so using this method the frequencies are processed independently and do not affect one another.

3.5. Putting a full Fourier block together

You might react to the fact that this approach does not take frequency-to-frequency interactions into account. Additionally, as we will see later, we also only keep a small set of the lower frequencies for the mixing. So how does the network ever capture small local details (the high frequencies that we discard) or the nonlinear interactions between different frequencies?

To address this, we combine the Fourier branch K_ℓ with a local branch W_ℓ and a nonlinear activation σ, which in our network is GELU. Together, these form a Fourier block. If v_ℓ is the stack of feature sheets entering block ℓ, its output is

$$v_{\ell+1}(\mathbf{x}) = \sigma(W_\ell v_\ell(\mathbf{x}) + (K_\ell v_\ell)(\mathbf{x}) + b_\ell)\,.$$

Here, b_ℓ is a learned bias, as in an ordinary neural-network layer. Both branches produce values at grid points, so we add them and apply the activation in physical space. Chapter 4 follows how this works in the implementation. One way to see this is that each part of this equation covers something that the others don't take into account:

- K_ℓ (global, low-frequency): This part combines information from across the domain. We keep only the lower frequencies here, which limits how many matrices $R(\mathbf{k})$ we need to learn. For our fluid problem, this is also a reasonable starting point because viscosity tends to smooth out the finer variations. Those variations can still matter, though, which is one reason for including the other branch.
- W_ℓ (local, all-frequency): Because K_ℓ removes high frequencies, W_ℓ acts as a residual path in physical space. This part mixes the C channels at each pixel without removing any frequencies, so this gives the high-frequency information a path around the cutoff.
- σ (nonlinear activation): Up until now the Fourier branch has handled each frequency separately. GELU acts after we have combined the waves back into a field. It responds to their combined value at each point and therefore also takes the interaction between different frequencies into account. This effectively lets the network learn more complicated behaviour than simply making individual waves stronger, weaker or shifting them.

In the network we built, the first three blocks apply GELU at the end, and the fourth does not apply it before the final projection layer.

4. Architecture and implementation from scratch

The math can feel a bit abstract but in this chapter we will try to connect it to something more concrete. Here we will be following one input through our network and the tensor operations in our code.

4.1. Inputs and layers

Lifting (P). The input is a tensor of shape $(B, 64, 64, 12)$ during training. B is the batch size, or how many fluid trajectories we stack in one forward pass (think B boxes on top of each other). Each box contains 12 sheets, each with 64×64 values. The first 10 sheets contain normalised vorticity from the past 10 time steps, and the last two contain the x and y coordinates of each grid point (the same grid for every trajectory). If you pick a single grid point and look straight down through the sheets, you get a vector of 12 numbers, 10 values describing the vorticity history at that point and 2 values identifying the position on the domain (one x coordinate and one y coordinate). Each entry comes from one channel, or sheet.

A linear layer P is then applied at every grid point, using the same weights everywhere, to mix those 12 channels into 64 ($12 \to 64$). These 64 sheets give the network more features to work with. Each one is a different linear combination of the ten historical values and the coordinates. The lifting layer has not yet exchanged information between positions, but it has given the

following blocks more ways to represent the information at each point. The coordinate sheets also allow these features to depend on where we are in the domain. This is useful here because the external forcing varies across the square, so its effect depends on position even though the boundaries are periodic. Because the mix happens strictly down the channel stack at each point, the channel count does not depend on the height or width of the grid. On a finer grid, there are more positions at which to apply the map, but still 12 input channels and 64 output channels at each position.

Fourier blocks (each $K + W$). After the lifting layer, every grid point holds a 64-vector of learned features, but nothing has mixed information across different positions on the grid yet. The four Fourier blocks are where we combine information across the spatial domain. Each block passes the same 64 feature sheets through the two parallel branches introduced earlier that we add together, $K + W$.

***K* (the spectral branch).** As we saw in Chapter 3, a convolution lets us use the same interaction pattern over the sheets, and the Fourier transforms let us calculate it efficiently. Here we apply that idea to our 64 feature sheets. The FFT treats opposite edges of each sheet as connected, so the convolution wraps around the square, just as our fluid does. (Note: Domains with walls or other geometries would require additional treatment of the domain and boundary conditions.)

We run a two-dimensional FFT on each of the 64 sheets separately which describes each sheet as a combination of waves. After the transform, the channels remain, but the entries no longer refer to spatial and historical interactions, instead, we store one complex coefficient per frequency, that records how strong the wave is (its amplitude) and how it is shifted across the domain (its phase).

We do not keep all of these frequencies. As we introduced earlier (but worth repeating) viscosity dampens the high-frequency variations, which makes it reasonable to focus the global mixing on the lower modes. An additional benefit of this is keeping the number of learned matrices manageable (remember, we have one frequency matrix $R(\mathbf{k})$ per frequency $\mathbf{k}$). Each of these matrices mixes all 64 input channels into 64 output channels, so keeping all frequencies means both more weights to learn and more matrix multiplications in every forward pass. In our implementation, we keep two 12×12 blocks of low-frequency coefficients and set the remaining output coefficients to zero. This acts as a low-pass filter in the spectral branch. The reason for the two blocks is explained in the implementation note below.

So now we have 64 sheets where each point corresponds to one frequency. However it is important to note that the spatial grid points haven't individually become frequencies, instead we have changed how we represent the whole sheet. Now, choosing one frequency and looking down through all 64 transformed sheets, we get a 64-vector of complex coefficients. Each retained frequency has its own learned 64×64 complex matrix $R(\mathbf{k})$, which mixes those 64 numbers into 64 new ones. This is similar to the initial lifting layer, but here the channel mixing happens per frequency rather than per physical grid point. Complex weights allow the mix to both scale a wave's amplitude and shift its phase as channels are combined. Finally, `irfft2` (inverse FFT) reconstructs 64 real feature sheets on the spatial grid. Because each wave extends across the domain, modifying its coefficients mixes information globally across space.

***W* (the local branch).** Because K removes every frequency above those 12×12 blocks, using only K would discard the finer spatial variations in the input features. The local branch W provides a second path, almost like a residual connection, around that low-frequency cutoff so local details can still pass through the block.

Here, we take the same 64 real feature sheets that entered the block (before the FFT), pick a grid point, and look down through the sheets to get a 64-vector of real numbers. A learned

64×64 real matrix (implemented as a 1×1 convolution with a bias) mixes those 64 numbers into 64 new ones, using the same weights at every position. Like lifting, it mixes channels at a single point without looking at neighbouring points.

Both branches return 64 sheets on the spatial grid. We add their outputs point by point and channel by channel, combining global spectral interactions with local pointwise features. A GELU activation follows each of the first three blocks, and the fourth block has no activation at its output.

Projection (Q). After the four blocks, each grid point holds 64 learned feature values and we need to combine these into one predicted vorticity sheet, i.e., the field at the next time step. We could do this with a single weighted sum, but then each feature would have a fixed contribution, completely independent of the values of the others. Instead, we first form 128 different weighted combinations of the 64 features and apply GELU to each one. This lets the contribution of a feature depend on the other features it is combined with. The final layer then takes a weighted sum of these 128 responses to produce one output value. Together, these steps give $64 \to 128 \to 1$. The intermediate layer gives the network more ways to combine the information gathered by the Fourier blocks before making the final prediction. Note that using 128 intermediate features is an architectural choice, rather than a requirement of the operator formulation.

Stage	Operation	Spatial mixing	Channels per position
Input	10 past vorticity frames + (x, y) coordinates	None	12
Lifting (P)	Shared pointwise linear layer	None (pointwise)	$12 \to 64$
Blocks 1–3	GELU($K + W$)	Global (K) + pointwise (W)	$64 \to 64$
Block 4	$K + W$ (no activation)	Global (K) + pointwise (W)	$64 \to 64$
Projection (Q), hidden layer	Linear layer + GELU	None (pointwise)	$64 \to 128$
Projection (Q), output layer	Linear layer	None (pointwise)	$128 \to 1$

Implementation note on FFT storage and scaling. Recall that the pixel area ΔA in Section 3.2 kept the integral approximation from growing simply because we added more grid points. In the Fourier branch, the transform pair handles the grid-size scaling. Our forward FFT adds contributions without dividing by the number of grid points M, while the inverse FFT divides by M. This balances the larger sums on a finer grid, so we do not multiply by ΔA separately when applying the learned frequency weights.

In code, we use `rfft2`, an FFT specialised for real inputs. Because the input sheets are real, roughly half of the Fourier coefficients are redundant conjugates (Hermitian symmetry), so `rfft2` avoids storing the negative frequencies along the second transformed axis. In the remaining array, we apply our learned weights to two 12×12 blocks: low nonnegative frequencies and low negative frequencies along the first axis (which sit at opposite ends of that axis under standard FFT indexing), each paired with the lowest 12 nonnegative frequencies along the second axis.

4.2. Building a forecast

One forward pass produces a single predicted frame. To generate a 20-step forecast, we roll the model out autoregressively. This means that we predict the next frame, slide the 10-frame input window forward by dropping the oldest vorticity sheet and appending the new prediction (while leaving the (x, y) coordinate sheets unchanged), and repeat 20 times.

After the first prediction, the input contains nine observed frames and one predicted frame. After ten predictions, all ten frames in the input window are predictions made by the model. This also means that any mistakes can become part of the input used to make later predictions.

During training, we run this same 20-step autoregressive rollout rather than feeding in the true intermediate frames, propagating gradients through all 20 steps in a single backward pass. This trains the model on the consequences of feeding its own predictions back in, so errors that accumulate at later steps can influence how earlier predictions are made. Section 5 covers the loss function, normalisation, and evaluation metrics in detail.

4.3. Zero-shot resolution transfer

Because the lifting layer P, the local branch W, and the projection Q all act strictly on channel vectors at individual points, their weight dimensions do not depend on the number of grid points. The spectral branch can reuse its weights too. A wave that makes two full oscillations across the square still makes two oscillations when we sample it on a 256×256 grid instead of a 64×64 grid. We simply have more points describing the same wave. Its frequency label $\mathbf{k}$ stays the same, so we apply the same learned matrix $R(\mathbf{k})$. This works for each of the frequencies retained by the model, allowing us to run it on the finer grid without retraining.

Our normalisation also depends on position, so we need values for the new grid points that lie between the training points. We explain this adjustment and test the finer-grid predictions in Section 6.

5. Data, training and evaluation

5.1. Dataset and split

We use the Navier–Stokes dataset released with the original FNO paper by Li et al., available through their dataset collection. It contains 10,000 trajectories on a 64×64 grid.[2]

The training configuration loads the first 1,400 trajectories. A random permutation with seed 0 assigns 1,000 to training, 200 to validation and 200 to testing. We keep each trajectory entirely within one subset, so the model never trains on some frames of a trajectory and is then tested on other frames from that same trajectory. During training, we use the validation set to choose which saved version of the model to keep. We then evaluate that version on the test set to obtain the final results.

After loading, we arrange the data as (N, H, W, T): trajectory, two spatial axes (Height, Width) and time. The separate 256×256 dataset stores the spatial axes in the opposite order, so we swap them when loading it. This makes sure the model sees the same coordinate directions during training and evaluation.

[2]The file is named `ns_V1e-4_N10000_T30.mat`, but contains fifty saved frames per trajectory. We use only the first thirty, divided into ten inputs and twenty targets.

5.2. Normalisation

Before training, we normalise the vorticity values. At each position and saved time, we subtract the mean across the training trajectories and divide by their standard deviation. Subtracting the mean centres the values around zero. Dividing by the standard deviation measures how far a value lies from that mean compared with the usual variation at that position and time. A normalised value of about 2, for example, means roughly two standard deviations above the training mean. This helps prevent parts of the data with larger numerical values from dominating just because of their scale.

Writing the mean as μ and the standard deviation as s, the normalised field is

$$\widetilde{\omega}(t, \mathbf{x}) = \frac{\omega(t, \mathbf{x}) - \mu(t, \mathbf{x})}{s(t, \mathbf{x}) + \varepsilon}, \qquad \varepsilon = 10^{-5}.$$

Each frame uses the mean and standard deviation for its own saved time, rather than its position in the input window. For example, when frame 2 moves from the second input channel to the first, its normalised values stay unchanged. Predicted frames are also fed back directly in normalised form.

The model predicts these normalised values. To recover physical vorticity, we reverse the transformation using the mean and standard deviation for the corresponding target time. When increasing the resolution, we interpolate the training mean and standard-deviation fields spectrally onto the finer grid.

We actually stumbled on this while trying to make sense of some strange preliminary results. At first, we had only looked at the standard deviation in the first frame, where it was fairly similar across the field. There did not seem to be much spatial structure to take into account. When the results looked odd, we went back and plotted the standard deviation frame by frame in our data-statistics notebook. We saw some patterns straight away! By the tenth frame, there were clear bands across the domain. Looking only at the first frame had hidden how much these statistics changed as the fluid evolved.

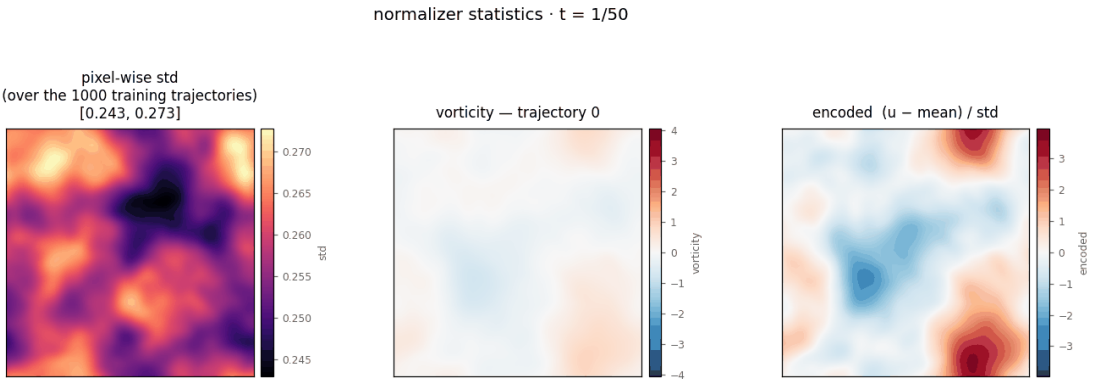


Figure 1: Normalisation statistics at the first saved frame

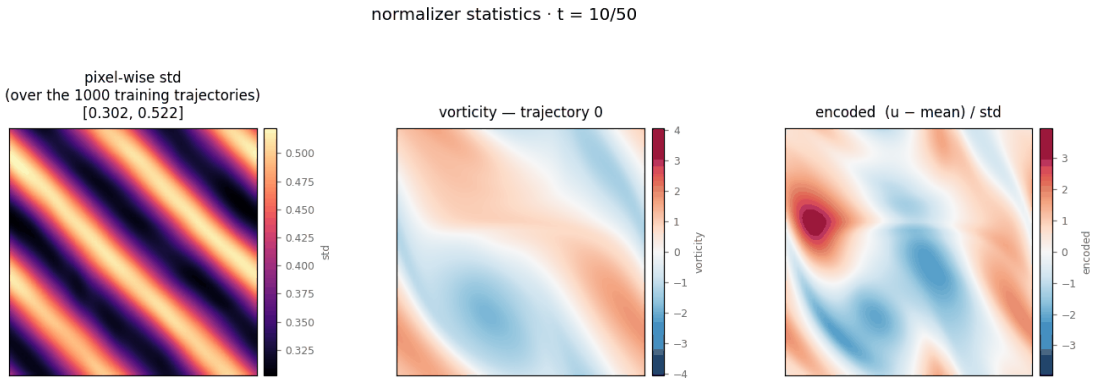


Figure 2: Normalisation statistics at the tenth saved frame

First and tenth saved frames from the normalisation-statistics animation. The left panel shows the standard deviation across the 1,000 training trajectories, the middle shows one trajectory, and the right shows its normalised values. The standard-deviation colour scale changes between frames. At the first frame, it spans only about 0.243–0.273, while at the tenth it spans about 0.302–0.522 and shows clear bands. The colours in the first panel therefore make a relatively small variation look more pronounced.

5.3. Loss and metrics

We compare each prediction with the corresponding frame from the dataset, which serves as the ground truth, or label. Here that ground truth comes from a numerical simulation. For one predicted frame p and ground-truth frame y, the relative L_2 error is

$$e(p, y) = \frac{\|p - y\|_2}{\|y\|_2} = \frac{\sqrt{\sum_j (p_j - y_j)^2}}{\sqrt{\sum_j y_j^2}},$$

where j corresponds to grid points. We first subtract the ground-truth value from the prediction at every point. Squaring these differences prevents positive and negative errors from cancelling. We add the squares and take the square root to get one measure of the overall difference between the fields. The denominator measures the size of the ground-truth field in the same way.

The choice of norm also affects what the model focuses on correcting first. With L_2 ($\|\cdot\|_2$), a few large mistakes count more than the same total absolute error spread across many small mistakes. For example, errors of 2 and 0 give a larger L_2 error than errors of 1 and 1, even though their absolute errors add up to the same amount.

Dividing by that size makes the error relative. The same absolute error may be small compared with a strong vorticity field but large compared with a weak one, so we want our loss to account for that difference. (Note: An error of 0.1 means the overall difference has one tenth of the ground-truth field's norm. It does not mean that every grid point has a ten-percent error.)

Training sums this error over all twenty predicted frames in normalised space and performs one backward pass through the rollout. We measure the error over the twenty-frame forecast in two ways:

- **Step error:** calculate the relative error separately for each predicted frame, then average over the twenty frames and the evaluated trajectories.

- **Full error:** stack the twenty frames, calculate one relative error for that whole forecast, then average over trajectories.

As the model feeds its predictions back into the input, an earlier mistake can affect later predictions, and these accumulated errors are included in both metrics. Step error gives each frame equal importance after measuring its relative error, while full error treats the whole forecast as one large collection of values. For the full error, frames with larger ground-truth field magnitudes have more influence.

5.4. Training setup

Setting	Value
Training / validation / test trajectories	1,000 / 200 / 200
Input / forecast frames	10 / 20
Batch size	32
Optimiser	Adam
Initial learning rate	10^{-3}
Weight decay	10^{-4}
Learning-rate schedule	Halve every 100 epochs
Epochs	500
Seed	0
Checkpoint selection	Lowest validation full error

Training ran on the Aixia cluster (AiQu), using one NVIDIA A100 with 80 GB GPU memory, 128 GB system RAM and six CPU cores. The container used CUDA 12.6.2 and PyTorch 2.5.0. Five hundred epochs took 69.5 minutes, approximately eight seconds per epoch.

The training and validation errors decrease quickly and then flatten. We select the checkpoint from epoch 491, where the validation full error is lowest at approximately 0.211 in normalised space. Section 6 reports the test results after converting the predictions back to physical vorticity.

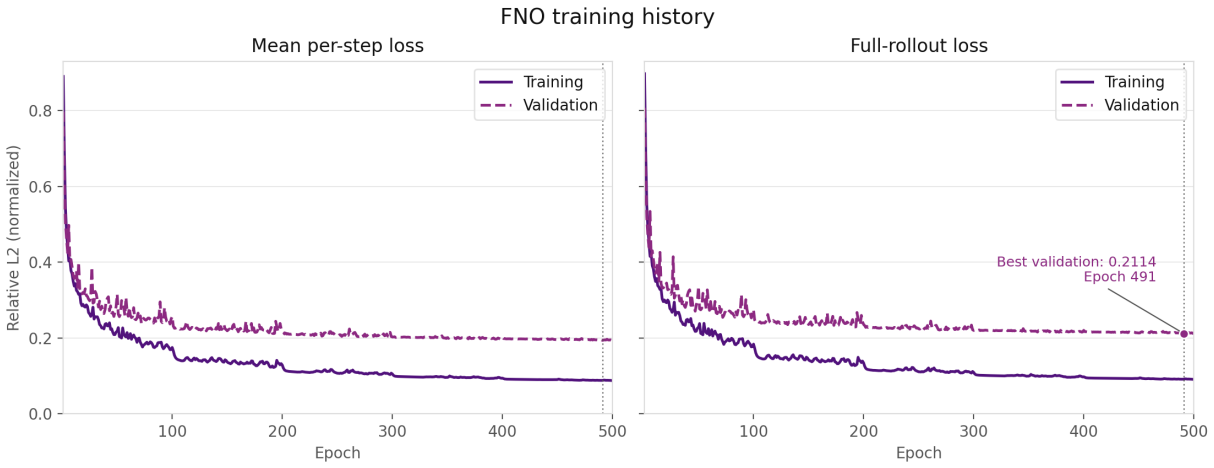


Figure 3: Training and validation relative L2 errors over 500 epochs

Training and validation errors in normalised space. The left panel shows mean per-step error and the right shows full-rollout error. The selected checkpoint is marked at epoch 491 (epoch 490 in the zero-based training log), where validation full error is approximately 0.2114.

6. Results

We looked at how accurately the network predicts the next twenty fluid states, how long those predictions take compared with the numerical solver, and whether the same model works on a finer grid. To make the timing comparison fair, we ran the FNO and numerical solver on the same machine.[3] The times below cover predicting the twenty future frames.

6.1. Accuracy

We first tested the model on 200 fluid trajectories that it had not seen during training. For each trajectory, we used ten observed frames as input and predicted the next twenty. Each prediction was then fed back into the input to make the following prediction.

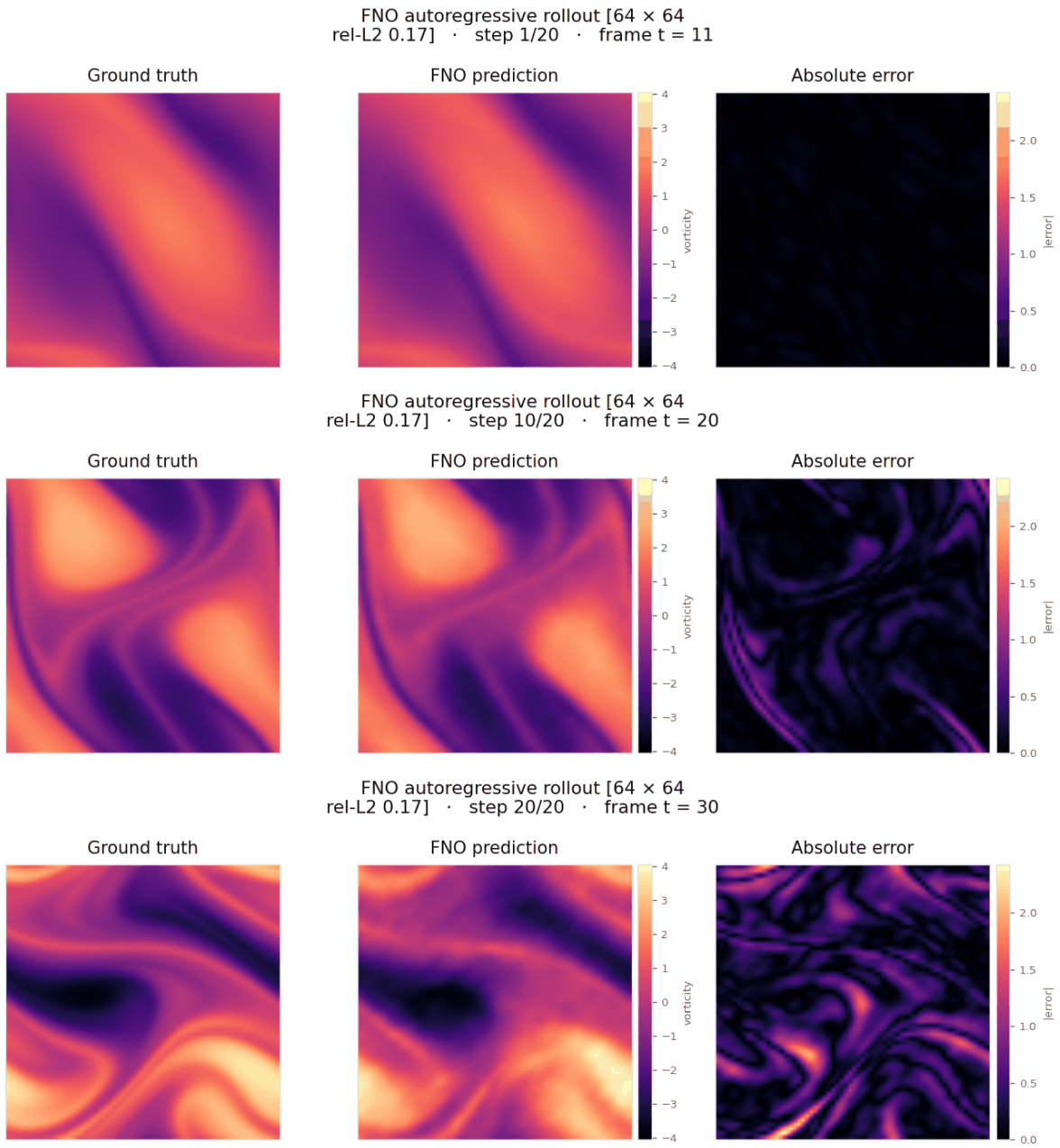
	Mean step error	Full-forecast error
FNO	0.1422	0.1813

These are the relative L_2 errors from Chapter 5, measured on physical vorticity after reversing the normalisation. [4]

The visualisations below give us a way of examining these errors and how they change over the rollout. The error panels make the differences between the prediction and the label easier to spot. In this example, those differences become larger later in the rollout.

[3]The timing experiments ran in a laptop VM (Lima Ubuntu on an M1), with `OMP_NUM_THREADS=1`, eager execution and float32. Training used the A100 described in Section 5.

[4]Li et al. (2021) report a relative L_2 error of 0.1559 for FNO-2d at this viscosity, forecast horizon and training-set size of 1,000. They do not specify whether this is per-step or stacked, and their script logs both, so a direct comparison with either column is uncertain. Our errors are of the same order of magnitude.



Example of the twenty-frame autoregressive forecast. The errors in the table are measured across the complete test set.

6.2. Speed versus accuracy

As we saw in Chapter 2, the numerical solver advances through many small steps between saved frames. Taking larger steps makes it faster, but also changes its accuracy, and eventually the simulation becomes unstable. So the speed advantage of the FNO depends partly on which solver step we compare it with.

The table below compares three solver step sizes with the FNO on the 64×64 grid. The largest stable step we tested was $\Delta t = 1.6 \times 10^{-3}$. The FNO predicts each saved frame in one pass, using twenty passes for the whole forecast. The FNO times are averages per forecast when using batches, as explained in Section 6.4.

Method	Full-forecast error	Time per forecast (s)	FNO speedup
Solver $\Delta t = 1.6 \times 10^{-3}$	0.0248	1.203	5.3×
Solver $\Delta t = 10^{-3}$	0.0211	1.931	8.6×
Solver $\Delta t = 10^{-4}$	0.0155	19.875	88×
FNO	0.1813	0.225	—

Compared with the largest stable step we tested, the FNO was about five times faster. Compared with the much smaller step used to generate the data, it was 88 times faster. Reducing the solver's step size increased its runtime as it forced it to take more steps.

Of course, the numerical solver was also considerably more accurate in both cases.[5] Whether the faster prediction is useful therefore depends on how much error we can accept for the task. Appendix C gives the full measurements and implementation details.

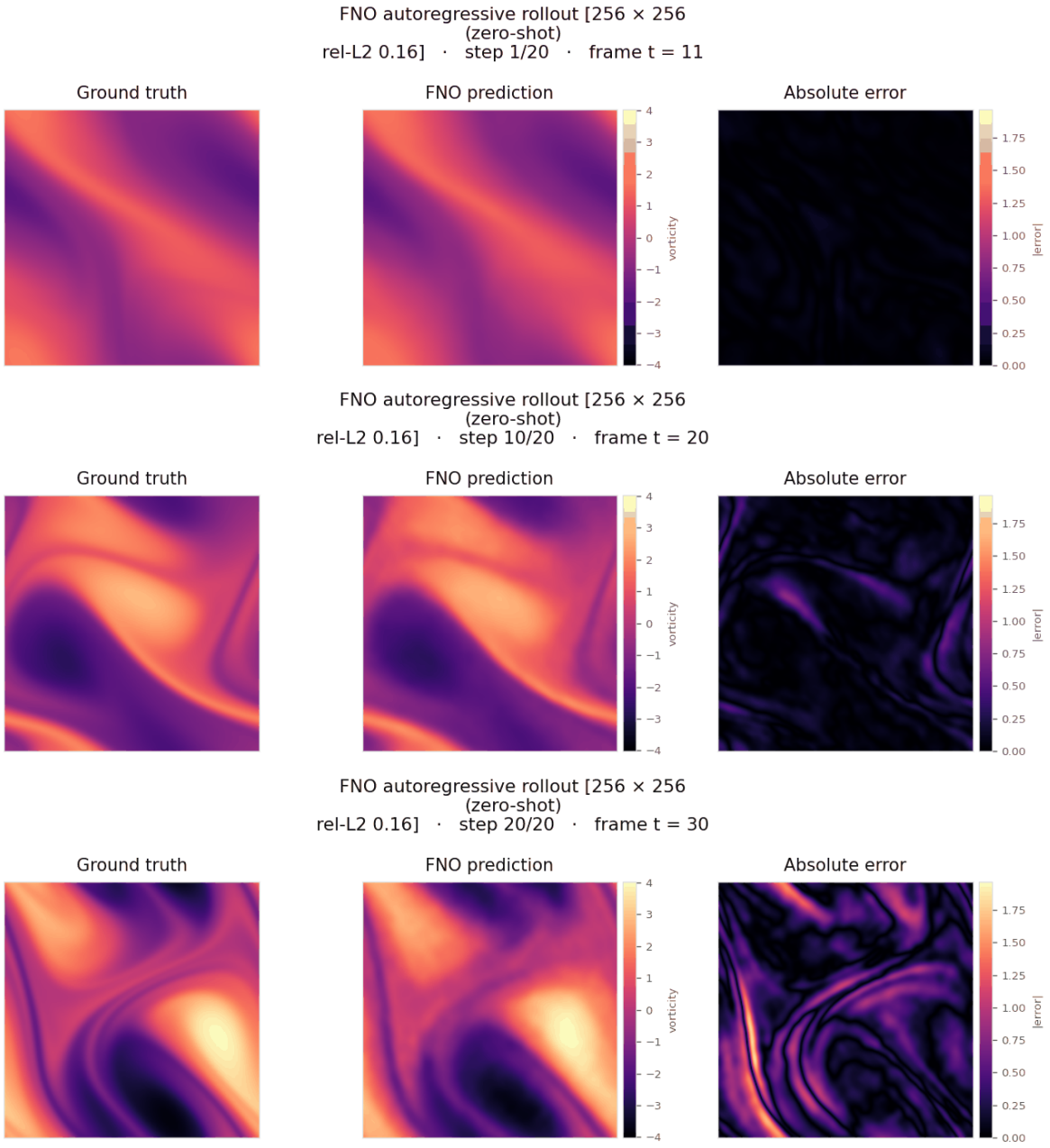
6.3. Transfer to 256×256 without retraining

One of the ideas that motivated this project was that the learned weights should not depend on how finely we sample the field. We therefore took the network trained on a 64×64 grid and evaluated it on a 256×256 grid. That gives us sixteen times as many points to predict, using the same weights and no additional training! This is what is meant by zero-shot resolution transfer.[6]

Across the ten trajectories, the mean step error was 0.145 with a standard deviation of 0.061, indicating how much the error varied between trajectories. The mean is close to the 0.142 measured on the coarser grid. This is encouraging, although we only tested ten trajectories here, and they were different from the 200 used in the earlier evaluation. We therefore cannot conclude that changing the resolution leaves accuracy unchanged.

[5] Even at $\Delta t = 10^{-4}$, the 64×64 solver has a mean step error of about 0.012. The ground-truth test data were downsampled from a finer grid, so using the same time step on the coarser grid does not reproduce them exactly.

[6] As described in Section 5, we evaluate the training mean and standard-deviation fields on the finer grid using their Fourier representation, and preserve the coordinate convention used during training.



Example prediction on the finer grid using the weights trained at 64×64.

We also compared the prediction times on these ten trajectories. The largest stable solver step we tested on this grid was $\Delta t = 8 \times 10^{-4}$.

Method	Mean step error	Time per forecast (s)	FNO speedup
Solver $\Delta t = 8 \times 10^{-4}$	0.0093	51.165	13×
FNO	0.145	3.869	—

On the finer grid, the FNO took about 3.9 seconds per forecast, compared with about 51 seconds for the largest stable solver step we tested. This gives a speedup of roughly thirteen times. Both methods became slower on the finer grid, but the numerical solver's cost increased more in this experiment. Its predictions were still considerably more accurate. Appendix C includes a separate single-trajectory comparison using the smaller solver step that generated the data.

6.4. Batching

There is one practical detail to add to the reported times. The network can process several trajectories together in a batch, which lets some of the work be shared. We did a batching sweep with batch sizes of 1, 4, 8 and 16. Predicting one trajectory on its own took about 344 ms. A batch of eight brought the average down to about 224 ms per trajectory, but of course, the whole batch took longer to finish.

The FNO timings in the comparison tables use the average time per trajectory when processing a batch. Appendix B looks more closely at the batching sweep, why batching helps and where the computation time goes.

7. Discussion and conclusion

We started this project because we wanted to understand operator learning, and specifically Fourier neural operators, well enough to build one ourselves. The mathematics can feel quite abstract at first, especially the idea of learning a mapping between functions. Our aim with this report has been to connect that idea to familiar concepts and explain how the different parts of the network work together.

Our implementation learned to predict twenty future fluid states from ten observed frames. At 64×64, it was about five times faster than the numerical solver using the largest stable time step we tested, with predictions processed in batches. However, the solver was considerably more accurate. Comparing against a smaller solver step increased the measured speedup to $88\times$. How useful that speedup is depends on the accuracy we actually need in an application. As discussed in the introduction, the cost of generating training data and training the network also has to be justified by making enough predictions afterwards.

We could also run the same learned weights on a 256×256 grid without retraining. This was a useful way to see the operator idea in practice. Increasing the number of sample points did not require larger weight matrices or a new network. The mean step error was close to that on the coarser grid, although the finer-grid evaluation used only ten trajectories from a different test set.

Building the model also showed us how much the details around the architecture and data matter. Our early look at the first frame suggested that the normalisation statistics were fairly uniform across the field. It was only after investigating strange preliminary predictions and looking through later frames that we noticed clear patterns.

There is still plenty we would like to explore. Longer rollouts would show how far we can keep feeding predictions back into the model before the errors become too large. Testing different forcing patterns or viscosities would explore conditions beyond those used here. For an engineering application, we would also need to check whether the predictions are good enough for that task, rather than relying only on the field error.

We hope this report gives you enough intuition to explore these questions further and a starting point for building neural operators yourself.

Acknowledgements

We thank Aixia for providing access to the AiQu computing resources used for training.

Appendix A. How the numerical solver works

Here we work through the solver from Section 2 in more detail. It explains why viscosity is cheap to handle in Fourier space and why the explicit advection update restricts the time step.

A.1. Why Fourier space simplifies viscosity

For two-dimensional velocity $\mathbf{u} = (u_1, u_2)$, scalar vorticity is $\omega = \partial u_2/\partial x_1 - \partial u_1/\partial x_2$. We use complex numbers here as a compact way to describe waves. The identity $e^{i\theta} = \cos\theta + i\sin\theta$, with $i^2 = -1$, packages a cosine and a sine into one expression. A complex coefficient can be drawn as an arrow in a plane. Its length represents amplitude and its angle represents phase, so changing the angle shifts the wave.

On the periodic unit square, a Fourier mode $k \in \mathbb{Z}$ is the wave $e^{2\pi ikx}$, so derivatives bring factors of $2\pi k$. In two dimensions the wavevector is $\mathbf{k} = (k_x, k_y)$.

The traditional solver uses a semi-implicit (IMEX) split: viscosity is treated implicitly and advection explicitly.

- **Implicit (looking forward): viscosity.** Solved with Crank–Nicolson, which means calculating the viscous contribution from a 50/50 mix of where the fluid is now and where it *will be* in the next step. Doing this for viscosity is easy and cheap in Fourier space (it is just division). The derivation is a little tricky, but the useful picture is what differentiation does to a wave. In physical space, viscosity works against momentum between neighbouring grid points, so advancing in time would mean inverting a large coupled matrix. Transforming to Fourier space turns differentiation into scaling each wave by a factor $-|2\pi k|^2$:

$$\frac{d^2}{dx^2} e^{2\pi ikx} = (2\pi ik)^2 e^{2\pi ikx} = -|2\pi k|^2 e^{2\pi ikx}.$$

By definition, an eigenfunction of an operator $\mathcal{L}$ is any non-zero ϕ with $\mathcal{L}[\phi] = \lambda\phi$. So $-|2\pi k|^2$ is the eigenvalue of the Laplacian on that wave, and each mode can be updated on its own.

- **Explicit (using the present state): advection.** Solved using only the current state. Calculating advection in Fourier space requires a convolution over pairs of modes, so the solver inverse-transforms, multiplies in physical space, and transforms back.

A.2. The time-step update

The time-step balance approximates the vorticity equation from Section 2: the change in vorticity combines transport, viscous smoothing and forcing. Advection and forcing are treated **explicitly** (evaluated only at the known present state, n). Viscosity is treated **implicitly** with Crank–Nicolson, i.e. a 50/50 average of the viscous contribution at the present state n and the unknown future state $n+1$.

Hats are Fourier coefficients, $\hat{\omega} = \mathcal{F}[\omega]$ (the amplitude and phase information for each wave). $\hat{F}(\omega^n)$ is the advection term. The discrete time-step balance is:

$$\frac{\hat{\omega}^{n+1} - \hat{\omega}^n}{\Delta t} = \underbrace{-\hat{F}(\omega^n) + \hat{f}}_{\text{explicit}} \underbrace{- \tfrac{1}{2}\nu|2\pi\mathbf{k}|^2\big(\hat{\omega}^{n+1} + \hat{\omega}^n\big)}_{\text{implicit (Crank–Nicolson)}}.$$

Multiply everything by Δt to clear the denominator, then move all the unknown future terms ($\hat{\omega}^{n+1}$) to the left and group the known present terms ($\hat{\omega}^n$) on the right:

$$\hat{\omega}^{n+1}\Big(1 + \tfrac{1}{2}\Delta t\,\nu|2\pi\mathbf{k}|^2\Big) = \hat{\omega}^n\Big(1 - \tfrac{1}{2}\Delta t\,\nu|2\pi\mathbf{k}|^2\Big) - \Delta t\,\hat{F}(\omega^n) + \Delta t\,\hat{f}.$$

Divide by the bracketed coefficient on the left to isolate the future state. The implicit solve in Fourier space is just scalar division, once per mode:

$$\hat{\omega}^{n+1} = \frac{-\Delta t\,\hat{F}(\omega^n) + \Delta t\,\hat{f} + (1 - \tfrac{1}{2}\Delta t\,\nu|2\pi\mathbf{k}|^2)\hat{\omega}^n}{1 + \tfrac{1}{2}\Delta t\,\nu|2\pi\mathbf{k}|^2}.$$

This is the integrator that generated the training data, using $\Delta t = 10^{-4}$. Saved frames are one time unit apart, so generating a 20-frame forecast takes 200,000 substeps at that setting.

A.3. A geometric picture of explicit-step instability

To picture what can go wrong with an explicit step, consider a single sine wave carried along at a constant velocity u in one dimension. Its shape should stay the same as it moves. A shift of this wave is the same as changing its phase. In the complex plane the coefficient $\hat{\omega}$ is a point at distance A from the origin and changing phase means that point runs around a circle. It does not get closer or farther.

How fast it runs is $2\pi ku$ (angular speed, on $[0, 1]$): a finer wave (large k) or a faster flow (large u) means the pattern repeats more often as it slides, so the phase winds faster. The exact step of length Δt is a short arc on that circle.

An explicit step follows the tangent in a straight line,

$$\hat{\omega}^{n+1} = \hat{\omega}^n - i\,2\pi ku\,\Delta t\,\hat{\omega}^n.$$

Even a short tangent step lands slightly outside the circle:

$$|\hat{\omega}^{n+1}| = \sqrt{1 + (2\pi ku\,\Delta t)^2}\,|\hat{\omega}^n|,$$

so the explicit piece on its own always grows a little. A long one shoots well outside: the ripple moves and grows taller, then grows again at the next step.

The explicit advection update therefore introduces artificial growth. Viscous damping can counteract it for sufficiently small steps, but the balance depends on viscosity, velocity and the time-stepping scheme. This picture assumes a fixed velocity. Establishing stability for the full nonlinear solver requires more analysis.

A common way to express the advective step-size constraint is $\Delta t \cdot \max|\mathbf{u}| \cdot 2\pi k_{\max} \lesssim C$, where C depends on the scheme. For the explicit-Euler advection illustrated above, pure advection grows at every nonzero step size, so this condition alone does not establish stability. We use the measured stable and divergent runs in Section 6 to describe the practical limit for our solver.

Appendix B. Batching and profiling

We also timed requests to the running server at different batch sizes ($n = 100$ requests per cell, medians):

batch	p50 ms	samples / s	ms / sample
1	344	2.90	344
4	1059	3.77	265
8	1794	4.45	224
16	3669	4.36	229

The 225 ms / traj in the solver table is the same 20-step rollout as the 224 ms/sample at batch 8 here. Batch 1 is 344 ms, which looks inconsistent until you notice that in-process batch 1 is already 343 ms. The extra ~ 120 ms comes from running the model on one trajectory at a time.

With `OMP_NUM_THREADS=1` throughout, batching lets several trajectories share the work of preparing the weights: you pay once to get the operator into the layout PyTorch wants, and then you apply it to several fields at once.

For this FNO, getting the operator into that layout is surprisingly expensive. The spectral mix is an `einsum` over the retained modes, which PyTorch turns into a batched matmul plus a gather. Concretely it copies 12×12 tiles of size 64×64 in each layer, which is 1,152 `aten::copy_` calls per step and 23,040 over the whole 20-step rollout. Those copies are fixed by the architecture (12 modes, 4 layers). They do not get cheaper when B grows. The FFT and the matmul just have more fields to work on.

We profiled 4 of the 20 rollout steps (each step executes the same network operations):

	batch 1 ($\sim$ 17 ms / step)	batch 8 ($\sim$ 90 ms / step)
memory (mostly those copies)	43%	18%
FFT	24%	34%
spectral mix (`bmm`)	20%	20%
pointwise (conv, Linear, GELU)	12%	25%

At batch 1, memory operations take the largest share of the roughly 17 ms per step. At batch 8, a step takes about 90 ms. The copy count stays the same, but there are more fields to transform and multiply, so FFTs now take the largest share. Which operation takes the most time depends on the batch size.

The batch timings show little improvement beyond batch 8. Sharing the setup across 8 trajectories takes you from 344 to 224 ms/sample (1.53×). Batch 16 is 229 ms/sample, so it gives no further improvement: once the copies are a small fraction of the time, adding more fields increases the FFT work while the gather cost is shared across the batch. If you write the HTTP times as some fixed cost plus a cost per sample, the fixed part comes out around 140 ms, which is pretty close to 43% of the 343 ms in-process batch-1 run ($\approx$ 147 ms). That simple model predicts more improvement at batch 16 than we actually measured.

Appendix C. Detailed solver comparisons

These are the full measurements behind Section 6. Errors are physical relative L_2 errors. Times are in milliseconds per trajectory for the twenty-frame forecast. The steps column counts numerical solver substeps or FNO forward passes, respectively.

C.1. The 64×64 comparison

	steps	rel-L_2 step	rel-L_2 full	ms / traj	FNO faster by
solver $\Delta t = 1.6 \times 10^{-3}$	12,500	0.0168	0.0248	1203	5.3×
solver $\Delta t = 10^{-3}$	20,000	0.0147	0.0211	1931	8.6×
solver $\Delta t = 10^{-4}$	200,000	0.0117	0.0155	19,875	88×
FNO	20	0.1422	0.1813	225	N/A

At 64×64, $\Delta t = 1.6 \times 10^{-3}$ was the largest stable solver step we tested, while 2.0×10^{-3} diverged. Against the largest tested stable step, the FNO was 5.3× faster, with about 8× larger mean step error and 7× larger full error. The held-out test accuracy was also reproduced through the HTTP server.

C.2. The 256×256 comparisons

	steps	rel-L_2 step	ms / traj	FNO faster by
solver $\Delta t = 8 \times 10^{-4}$ ($n = 10$)	25,000	0.0093	51,165	13×
solver $\Delta t = 10^{-4}$ ($n = 1$)	200,000	0.000002	381,167	110× (vs FNO $n = 1$)
FNO ($n = 10$)	20	0.145	3869	N/A
FNO ($n = 1$, same trajectory as slow solver)	20	0.249	3472	N/A

The ten-trajectory comparison gives a 13× speedup, with FNO mean step error about 16× larger than the solver's. The separate single-trajectory comparison uses $\Delta t = 10^{-4}$, the same scheme and time step used to generate that dataset. The solver therefore reproduces that trajectory essentially exactly. On that same trajectory, the FNO has step error 0.249 and takes 3472 ms, giving a speedup of about 110×. This single-trajectory result is separate from the ten-trajectory average.

Going from 64×64 to 256×256 increased the measured FNO time by about 17× and the solver time by about 43×, when comparing the largest tested stable solver steps at each resolution.

C.3. Implementation details

The solver uses complex `fft2` on real fields and has not been extensively optimised. Real-valued transforms and other optimisations could reduce its cost. Both implementations run with one thread, so the measured ratios should not be read as a general comparison with all numerical solvers.

The finer-grid experiments use the offline evaluation code. The HTTP server accepts only 64×64 inputs because its packaged normaliser stores statistics on that grid.

References and further reading

1. Li, Z., et al. (2021). *Fourier Neural Operator for Parametric Partial Differential Equations*. ICLR 2021; first posted in 2020. The original FNO paper and the architectural starting point for this project.
2. Kovachki, N., et al. (2023). *Neural Operator: Learning Maps Between Function Spaces*. Journal of Machine Learning Research, 24(89). A formal treatment of neural operators and their approximation properties.
3. Duruisseaux, V., Kossaifi, J., and Anandkumar, A. (2025; revised 2026). *Fourier Neural Operators Explained: A Practical Perspective*. A detailed guide to mathematical background, implementation and extensions. Section 3.8 surveys applications; Sections 2 and 3 develop the foundations and architecture.
4. Original FNO authors' dataset collection. Source of the Navier–Stokes training and evaluation data used here.